UNIVERSITÀ DEGLI STUDI DI BRESCIA · UNIBS.it

AA 2026/27

PAJC · PROGRAMMAZIONE AVANZATA IN JAVA E C

LC-01

# Warm-up C: la memoria in mano

Puntatori, array e stringhe, stack e heap, struct, union e campi di bit, il preprocessore e le macro




**prof. Massimiliano Redolfi**

Università degli Studi di Brescia · Ingegneria Informatica

# Oggi

1. **Perché il C**: ciò che Java fa al posto nostro
2. **Dal sorgente all'eseguibile**: strumenti, più file, `make`
3. **Funzioni e memoria**: il frame, i parametri copiati, i byte
4. **Puntatori**: `&` e `*`, `NULL`, `const`, puntatore a puntatore
5. **Array e stringhe**: `a[i]` è `*(a + i)`, l'array che decade, VLA, `string.h`
6. **Stack e heap**: `malloc` e `free`, i quattro errori, ASan
7. **struct, union e campi di bit**: `typedef`, `->`, chi possiede
8. **Preprocessore e macro**: parentesi, doppia valutazione, `do { } while (0)`
9. Errori frequenti e il C in due righe; lungo la strada, esercizi da 5–10 minuti

# 01

# Perché il C



# Ciò che Java fa al posto nostro

| In Java | In C |
|---|---|
| il garbage collector libera gli oggetti | `free`, a mano, una volta sola |
| un riferimento: non si somma, non scade | un puntatore: si somma, può puntare a memoria morta |
| `String` sa quanto è lunga | `char[]` chiuso da `'\0'` |
| `ArrayList` cresce da sola | `malloc`, `realloc`, e la lunghezza la tenete voi |
| un'interfaccia, una lambda | un puntatore a funzione (fra una settimana) |
| un'eccezione | un valore di ritorno da controllare |

Il C è il **contro-canto** di Java: ogni costrutto lo vedremo accostato a ciò che Java nasconde. È così che si capisce davvero che cosa costa un `new`.

# Lo stesso programma

**JAVA**

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Ciao, " + args[0])
    }
}
```

```
javac Hello.java && java Hello mondo
```

**C**

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("Ciao, %s\n", argv[1]);
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -o hello hello.c &&
./hello mondo
```

# La riga di compilazione

```
gcc -std=c17 -Wall -Wextra -Werror -g -o hello hello.c
```

- `-std=c17`: lo standard del corso; niente estensioni, niente C89 con le dichiarazioni in testa
- `-Wall -Wextra`: tutti gli avvisi. Un avviso in C è quasi sempre un bug che il compilatore ha visto prima di voi
- `-Werror`: gli avvisi diventano errori. Nei `Makefile` del corso c'è sempre
- `-g`: i simboli per il debugger e per i messaggi di AddressSanitizer
- `-fsanitize=address`: la rete di sicurezza per la memoria, la vedremo fra poco

> Diamo per scontati tipi, cicli, funzioni, array statici, `printf` e `scanf`. Da qui in avanti: la memoria.

# 02

# Dal sorgente all’eseguibile



# Gli strumenti, per sistema

| | Che cosa serve | Come si installa |
|---|---|---|
| **macOS** | `gcc` (è clang) e `make` | `xcode-select --install` (Command Line Tools) |
| **Linux** | gcc, make, gdb | Ubuntu e Debian: `sudo apt install build-essential gdb` |
| **Windows** | gcc e make dentro **WSL** | `wsl --install`, poi Ubuntu come sopra; in alternativa MSYS2 (`pacman -S mingw-w64-ucrt-x86_64-gcc make`) |

- editor: **VS Code** con l'estensione C/C++ e il terminale integrato; va bene anche CLion
- i comandi di oggi sono gli stessi sui tre sistemi, salvo dove è scritto. Su Windows senza WSL gli eseguibili finiscono in `.exe` e `make` può chiamarsi `mingw32-make`

# Con VS Code

Il repository del laboratorio ha la configurazione pronta (`.vscode/`), la guida `VSCODE.md` per i tre sistemi e uno script che controlla e installa i prerequisiti:

```
git clone https://github.com/max-ai4res/pajc-lab27.git
cd pajc-lab27 && ./setup-vscode.sh && code .
```

- estensioni: **C/C++** (sottolinea gli errori come `-Wall -Wextra`) e **CodeLLDB** (il debugger, uguale sui tre sistemi); su Windows anche **WSL**
- con `main.c` aperto: `⌘⇧B` / `Ctrl⇧B` fa `make`, **F5** compila e lancia il debugger, `make run` e `make asan` da *Terminal › Run Task*; gli errori nel pannello *Problems*
- il bottone ▷ «Run C/C++ File» compila senza i flag del corso: i task sì, lui no

# Quattro passi, un comando

```
hello.c  →  preprocessore  →  compilatore  →  assemblatore  →  linker  →  hello
            gcc -E             gcc -S          gcc -c           gcc
            (testo espanso)    hello.s         hello.o          + libc
```

- il **preprocessore** incolla gli `#include` e sostituisce le macro: lavora sul testo
- il **compilatore** traduce il C in assembly; l'**assemblatore** lo fa diventare codice macchina in un file oggetto `.o`
- il **linker** unisce i `.o` fra loro e con la libreria standard (`printf` sta lì): ne esce l'eseguibile
- `gcc -o hello hello.c` fa tutto in una volta; `gcc -c` si ferma al `.o`, ed è il passo che conta quando i file sono più di uno

# Un progetto in più file

```c
// rettangolo.h: le dichiarazioni, che cosa c'è
typedef struct { double x, y, w, h; } Rectangle;
double rectangle_area(const Rectangle *r);

// rettangolo.c: le definizioni, come funziona
#include "rettangolo.h"
double rectangle_area(const Rectangle *r) { return r->w * r->h; }

// main.c
#include "rettangolo.h"
```

```
gcc -std=c17 -Wall -Wextra -c rettangolo.c      # → rettangolo.o
gcc -std=c17 -Wall -Wextra -c main.c            # → main.o
gcc -o app main.o rettangolo.o                  # link
```

L'header lo includono entrambi: le chiamate del `main` sono controllate contro le dichiarazioni. Se cambia `rettangolo.c` si rifà **solo** il suo `.o`, e il link.

# `make`: una regola

```
app: main.o rettangolo.o
  gcc -o app main.o rettangolo.o

main.o: main.c rettangolo.h
  gcc -std=c17 -Wall -Wextra -c main.c

rettangolo.o: rettangolo.c rettangolo.h
  gcc -std=c17 -Wall -Wextra -c rettangolo.c
```

- **bersaglio**, due punti, le **dipendenze**; sotto, i comandi (la *ricetta*) con un **TAB** davanti, non spazi
- `make` costruisce il primo bersaglio (`make main.o` solo quello), e rifà un bersaglio se **manca** o se una dipendenza è **più recente**: cambiato `main.c`, solo `main.o` e `app`

# Variabili e regole con il pattern

```
CC     = gcc
CFLAGS = -std=c17 -Wall -Wextra -Werror -g
OBJ    = main.o rettangolo.o
app: $(OBJ)
  $(CC) $(CFLAGS) -o $@ $^
%.o: %.c rettangolo.h
  $(CC) $(CFLAGS) -c $<
clean:
  rm -f app $(OBJ)
.PHONY: clean
```

- `$@` è il bersaglio, `$<` la prima dipendenza, `$^` tutte; `%.o: %.c` è **una regola per tutti** i `.o`
- `clean` non produce un file: `.PHONY` lo fa eseguire sempre. `make clean && make` rifà tutto

# Il `Makefile` del corso, riga per riga

```
SRC = $(wildcard *.c)                       # tutti i sorgenti della cartella
BIN = $(patsubst %.c,bin/%,$(SRC))          # main.c -> bin/main, stack.c -> bin/stack

bin/%: %.c | bin                            # regola pattern; | bin: la cartella (mkdir -p) deve esse
  $(CC) $(CFLAGS) -o $@ $< $(LDLIBS)        # LDLIBS = -lm: la libreria matematica (serve su Linux)

run: bin/main                               # make run: compila se serve, poi esegue
  ./bin/main

asan: main.c | bin                          # make asan: lo stesso eseguibile con AddressSanitizer
  $(CC) $(CFLAGS) -fsanitize=address -o bin/main $< $(LDLIBS)
```

Ogni cartella `LezC01-*` ha questo `Makefile`: `make`, `make run`, `make asan`, `make clean`. L'eseguibile si chiama come il sorgente, in `bin/`, che git ignora; su Windows con MSYS2 `./bin/main.exe`.

# Prova · tre minuti con `make`

In `LezC01-Esempi`, dove i sorgenti sono nove e la regola `bin/%: %.c` li compila tutti:

1. `make`: compila tutti gli esempi in `bin/`
2. `make` di nuovo: «Nothing to be done»: niente è cambiato
3. cambiate un commento in `stack.c` e rilanciate `make`: si ricompila solo quello
4. `make clean`, poi `make bin/stack`: un bersaglio solo

> Se vedete `missing separator`: una ricetta è indentata con spazi invece del TAB. È l'errore numero uno con make.

# 03

# Funzioni e memoria



# Che cosa succede a una chiamata

```c
void incrementa(int n) {
    int doppio = n * 2;
    n++;
}
int main(void) {
    int n = 5;
    incrementa(n);          // n vale ancora 5
}
```

- il `main` ha il suo **frame** sullo stack: lì sta la sua `n`
- alla chiamata nasce un frame nuovo, **più in basso**: una **copia** di `n`, `doppio`, l'indirizzo a cui tornare
- `n++` scrive nella copia; al ritorno il frame sparisce e la `n` del `main` è intatta

# I due frame




- due `n`, due indirizzi: quella di `incrementa` sta 76 byte più in basso
- **per valore, sempre**: il C copia ciò che passate, anche una `struct` intera

# `dump`: la memoria byte per byte

```c
void dump(const char *nome, const void *p, size_t n) {
    const uint8_t *b = p;                          // la stessa memoria, un byte alla volt
    printf("%-8s %p  ", nome, p);
    for (size_t i = 0; i < n; i++) printf("%02X ", b[i]);
    printf("\n");
}

dump("n", &n, sizeof n);                           // n        0x16b6f2268  05 00 00 00
```

- `const void *` accetta l'indirizzo di qualunque cosa; dentro, un `uint8_t *` legge byte per byte
- `%p`: `0x16b6f2268` su macOS e Linux, `000000A5B7FFE4C8` su Windows, senza `0x`
- `frame.c` in `LezC01-Esempi` la usa su tutto

# `frame.c` dal vivo

```
$ make bin/frame && ./bin/frame
-- main, prima
n        0x16b6f2268  05 00 00 00
d        0x16b6f2260  00 00 00 00 00 00 F8 3F
s        0x16b6f225c  61 62 63 00
-- dentro incrementa
n        0x16b6f221c  05 00 00 00
doppio   0x16b6f2218  0A 00 00 00
n        0x16b6f221c  06 00 00 00
-- main, dopo
n        0x16b6f2268  05 00 00 00
```

- `5` occupa 4 byte, il meno significativo per primo; `"abc"` è `61 62 63` più lo **zero finale**; `1.5` è un `double` IEEE 754
- la `n` di `incrementa` ha un altro indirizzo: `n++` la porta a `06`, quella del `main` resta `05`

# Prova · prevedere prima di eseguire

```c
void raddoppia(int v[], int n) {
    for (int i = 0; i < n; i++) v[i] *= 2;
    n = 0;
}

int main(void) {
    int a[3] = {1, 2, 3};
    int n = 3;
    raddoppia(a, n);
    printf("%d %d %d  n=%d\n", a[0], a[1], a[2], n);
}
```

Che cosa stampa? Due minuti, poi la risposta la dà il compilatore, e la spiegazione la prossima sezione.

# 04

# Puntatori



# Una variabile ha un indirizzo

```
int x = 42;
int *p = &x;        // p contiene l'indirizzo di x
*p = 7;             // scrive nella cella puntata: x vale 7
```




`p` è una variabile come le altre: il suo valore è **l'indirizzo di** `x`, la freccia. `*p` è la cella in fondo alla freccia.

# `&` e `*`

```
int *p = &x;
printf("%p %d %d\n", (void *)p, *p, x);   // 0x16d366248 7 7
```

- `&x` è **l'indirizzo** di `x`: un numero, la posizione della cella in memoria
- `int *p` dichiara una variabile che **contiene un indirizzo** di `int`
- `*p` nell'espressione è **la cella puntata**: si legge e si scrive come fosse `x`
- `%p` stampa un indirizzo; il cast a `void *` è la forma corretta. Nelle slide gli indirizzi sono accorciati: `…6248` sta per `0x16d366248`

# Le due facce di `*`

```c
int *p;          // dichiarazione: p è un puntatore a int
*p = 3;          // espressione: la cella puntata da p
```

- nella **dichiarazione** `*` fa parte del tipo; nell'**espressione** dereferenzia
- `int *p, q;` dichiara un puntatore e un `int`: l'asterisco si lega al nome, non al tipo. Una dichiarazione per riga
- un puntatore non inizializzato contiene spazzatura: `*p = 3` scrive da qualche parte. Spesso un crash, a volte niente, mai un errore del compilatore

> `NULL` è il puntatore che non punta a nulla. Dereferenziarlo è un crash certo (*segmentation fault*), ed è il crash buono: si trova subito. Un puntatore **va inizializzato**, a un indirizzo o a `NULL`.

# Passare per riferimento

**JAVA**

```java
static void swap(int a, int b) {
    int t = a; a = b; b = t;
}

int x = 3, y = 7;
swap(x, y);          // x = 3, y = 7
```

I parametri sono copie: `swap` non può toccare `x` e `y`. Si scambiano i campi di un oggetto, mai due variabili

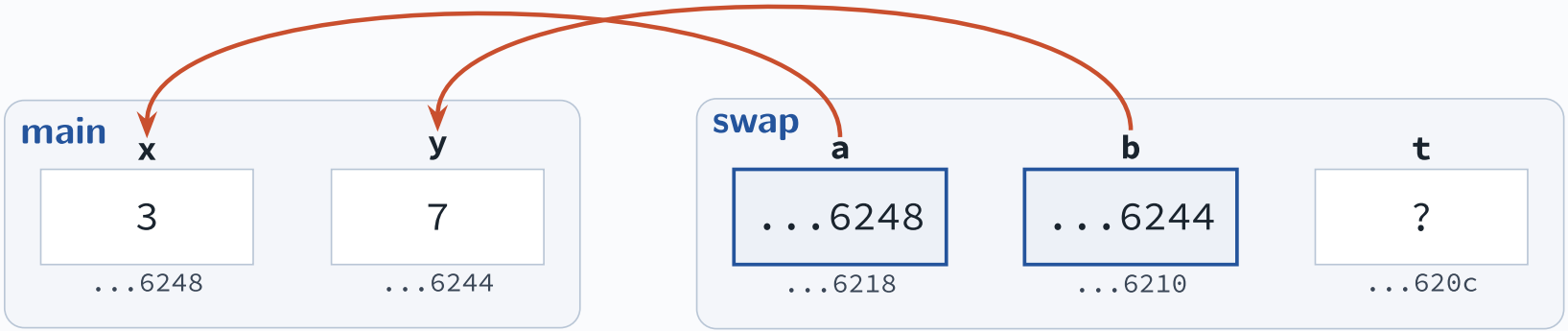
**C**

```c
void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

int x = 3, y = 7;
swap(&x, &y);        // x = 7, y = 3
```

Anche qui i parametri sono copie, ma copie di **indirizzi**: attraverso `*a` la funzione scrive nella `x` del chiamante

# Che cosa vede `swap`




- `a` e `b` sono copie, come in Java: ma copie degli **indirizzi** di `x` e `y`, e le frecce escono dal frame di `swap` per entrare in quello del `main`
- `*a = *b` segue le frecce: scrive in `x` il valore letto da `y`. In Java `a` e `b` conterrebbero `3` e `7`, e lo scambio resterebbe dentro `swap`

# `const`: che cosa è costante

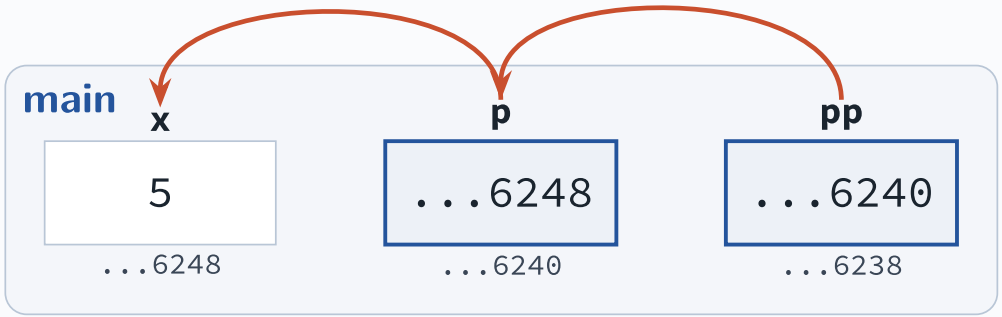
```
const int *p;           // la cella puntata: *p = 3 è un errore, p = &y va bene
int *const q = &x;      // il puntatore: q = &y è un errore, *q = 3 va bene
const int *const r;     // entrambi
```

- si legge **da destra a sinistra**: `p` è un puntatore a `int` costante; `q` è un puntatore costante a `int`
- `const T *` nei parametri è una promessa: **la funzione non modifica** ciò che riceve. `strlen(const char *s)`, `area(const Rectangle *r)`
- il compilatore la fa rispettare: scrivere attraverso `const` è un errore, non un avviso

> La regola pratica: ogni parametro puntatore che non va scritto è `const`. Il lettore sa che cosa cambia dopo la chiamata senza aprire la funzione.

# Puntatore a puntatore

```c
int x = 5;
int *p = &x;
int **pp = &p;      // pp contiene l'indirizzo di p
**pp = 9;           // x vale 9;  *pp = &y: ora p punta a y
```




- due frecce: `*pp` è `p`, `**pp` è `x`. `*pp = &y` cambia **dove punta** `p`
- serve a una funzione per **cambiare un puntatore** del chiamante

# Esercizio 1 · `swap` e dintorni

Aprite `LezC01-Swap/main.c`, poi `make run`. Il `main` stampa il risultato e il valore atteso.

1. `void swap(int *a, int *b)`: scambia le due variabili puntate
2. `void scambia_e_azzera(int *a, int *b)`: scambia e poi azzera `*b`; si può chiamare `swap`
3. se avanza tempo, `void swap_ptr(int **a, int **b)`: scambia **i puntatori**, non le variabili puntate

> Dieci minuti. Domanda da tenere a mente: perché il `main` passa `&x` e non `x`? E nella terza, perché `&px`?

05

# Array e stringhe



# Un array è un indirizzo, e una lunghezza che sapete voi

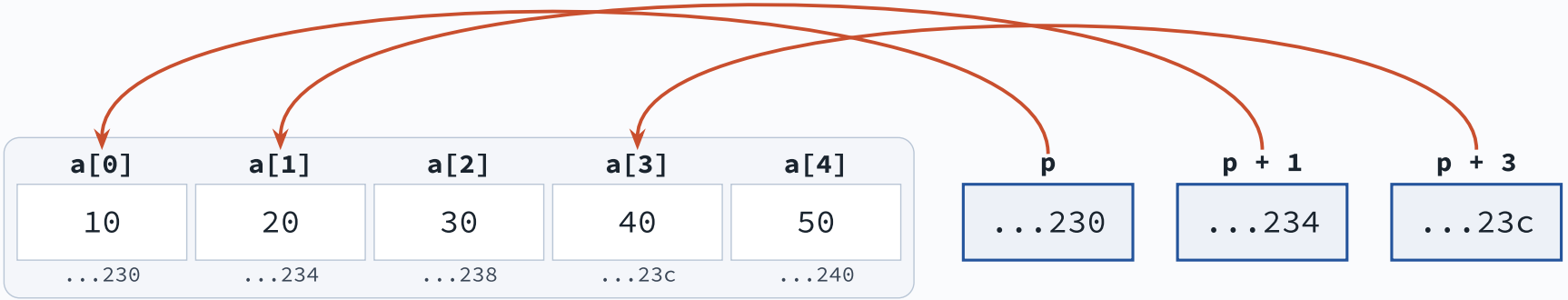
```c
int a[5] = {10, 20, 30, 40, 50};
int *p = a;             // a "decade" nell'indirizzo del primo elemento

a[2]  == *(a + 2)       // la notazione [ ] è aritmetica di puntatori
p[2]  == *(p + 2)       // e funziona anche sul puntatore
&a[2] == a + 2
```

- `a[i]` è **definito** come `*(a + i)`: le parentesi quadre sono zucchero
- il nome dell'array, in quasi ogni espressione, vale l'indirizzo del primo elemento
- il C non conosce la lunghezza: nessun `a.length`, nessun controllo su `a[7]`

# L'aritmetica avanza di `sizeof`

```c
int a[5] = {10, 20, 30, 40, 50};
int *p = a;
printf("%td\n", &a[4] - a);        // 4: la differenza conta elementi, non byte
```




- `p + 1` avanza di `sizeof(int)`, 4 byte: il tipo dice quanto è grande un passo; `(char *)p + 1` avanza di uno

# L’array decade: la dimensione viaggia a parte

```c
void stampa(int a[], size_t n) {          // int a[] qui vuol dire int *a
    printf("%zu\n", sizeof(a));           // 8: la dimensione del puntatore
    for (size_t i = 0; i < n; i++) printf("%d ", a[i]);
}

int v[5] = {1, 2, 3, 4, 5};
stampa(v, sizeof(v) / sizeof(v[0]));      // 20 / 4 = 5, qui funziona
```

- passato a una funzione l’array diventa **un puntatore**: `sizeof` dentro misura il puntatore, 8 byte
- `sizeof(v) / sizeof(v[0])` funziona solo **dove l’array è dichiarato**
- quindi ogni funzione su array riceve **anche la lunghezza**: `(int *a, size_t n)`

# Matrici

```c
int m[3][4];                    // 3 righe da 4 int, contigue: 12 int, riga dopo riga
m[1][2] = 7;                    // *(*(m + 1) + 2): la riga 1, poi la colonna 2

void stampa(int m[][4], size_t righe);   // le colonne devono essere note al compilatore
```

- una matrice statica è un **array di array**: un blocco solo, per righe (*row-major*)
- per `m[i][j]` il compilatore deve sapere quanto è larga una riga: nei parametri si omette solo la prima dimensione
- dimensioni decise a tempo di esecuzione: `malloc` di un blocco unico, indice `i * colonne + j`, oppure un VLA

# VLA: la dimensione a tempo di esecuzione

```c
size_t n = leggi();
int v[n];                       // array a lunghezza variabile: sullo stack, n * 4 byte
printf("%zu\n", sizeof v);      // calcolato a tempo di esecuzione
```

- **C99** li rende obbligatori, **C11** opzionali ( `__STDC_NO_VLA__` ), C23 tiene obbligatori solo i *tipi* VLA. MSVC non li ha mai avuti; il kernel Linux li ha tolti tutti nel 2018
- stanno **sullo stack**, piccolo e senza controlli: `./bin/vla 3000000` muore di *segmentation fault*, senza messaggio e senza un `NULL` da verificare
- niente inizializzatore, niente `static`

> Come variabile locale con `n` deciso dall'utente: no. Quella memoria si chiede con `malloc`.

# Il VLA buono: nei parametri

```c
double traccia(size_t n, double m[n][n]) {       // m: puntatore a righe di n double
    double t = 0;
    for (size_t i = 0; i < n; i++) t += m[i][i];
    return t;
}
double (*m)[n] = malloc(n * sizeof *m);           // n righe da n double, sullo heap
traccia(n, m);                                    // ./bin/vla 4 in LezC01-Esempi
```

- nessun array sullo stack: `m` è un **puntatore**; `n` viene prima nella firma perché il tipo di `m` la usa
- `m[i][j]` funziona per qualunque `n`: il compilatore calcola `i * n + j`. Prima si passava `double *m` e si moltiplicava a mano

# Il quiz d'esame

```c
int i[] = {2, 5, 7};
int *p = &i[*i];
int k = (*p)--;
```

Che cosa valgono `k`, `i[2]` e `*p`?

```c
int v[] = {1, 2, 3};
int *q = v;
int a = *q++;       // precedenza: *(q++)
int b = (*q)++;
```

E qui `a`, `b`, `v` e `*q`? La differenza fra `*q++` e `(*q)++` vale tre quesiti d'esame.

> Prima a mano, poi `LezC01-Esempi/quiz.c` per verificare.

# Le stringhe sono array di `char` chiusi da `'\0'`

```c
char s[] = "ciao";       // 5 byte: 'c' 'i' 'a' 'o' '\0', modificabile
char *t = "ciao";        // un puntatore a un letterale: sola lettura
printf("%zu %zu\n", sizeof(s), strlen(s));     // 5 4
```

- la lunghezza non è scritta da nessuna parte: `strlen` **cammina fino allo zero**. Senza lo zero cammina finché trova uno
- un letterale `"ciao"` sta in memoria di sola lettura: `t[0] = 'C'` compila e va in crash. `const char *t` dice la verità
- `s` è un array sullo stack, `t` un puntatore: `s = t` non compila, un array non si assegna

# `string.h` e le trappole

**JAVA**

```
String a = "ciao", b = "ciao";
a.equals(b)          // true
a = b;               // stesso riferimento
a.length()           // 4
a + "!"              // nuova stringa
```

In Java le stringhe sono immutabili e sanno la propria lunghezza: ogni operazione restituisce una stringa nuova e non c'è un buffer da dimensionare

**C**

```
char a[8] = "ciao", b[8] = "ciao";
strcmp(a, b) == 0   // uguali; a == b confront
strcpy(a, b);       // copia, se a è abbastanz
strlen(a)           // 4
strcat(a, "!");     // appende: ora servono 6
```

Ogni funzione si fida di voi: `strcpy` scrive finché trova `'\0'`, `strcat` idem. La dimensione del buffer la controllate voi, con `snprintf(buf, sizeof buf, ...)` quando potete

# Esercizio 2 · Tre funzioni su stringhe

Aprite `LezC01-Stringhe/main.c`, poi `make run`.

1. `void reverse(char *s)`: rovescia la stringa **sul posto**, senza array d'appoggio
2. `bool is_palindrome(const char *s)`: `radar` sì, `ciao` no; non modifica la stringa
3. `void freq(const char *s, size_t conta[26])`: quante `a`, quante `b`, …; `tolower` e `isalpha` stanno in `ctype.h`

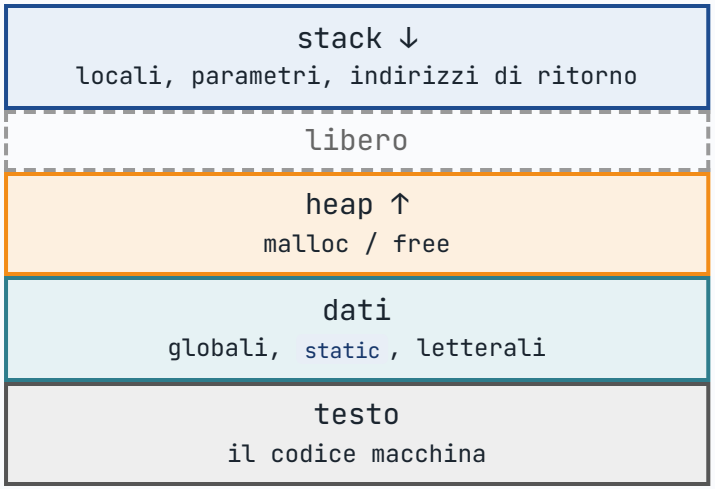
> Dieci minuti. Un indice dall'inizio e uno dalla fine bastano per le prime due; per la terza, `c - 'a'` è un indice.

# 06

# Stack e heap



# La memoria di un processo




- lo **stack** cresce a ogni chiamata e si ritira al ritorno: le locali **muoiono con la funzione**. Automatico, piccolo (8 MB)
- lo **heap** vive finché non lo liberate: la memoria di `malloc`. Ogni blocco ha un proprietario
- i **dati** durano quanto il programma
- Java ha le stesse zone, uno stack per thread e lo heap degli oggetti. La differenza è **chi libera**

# `stack.c` dal vivo: gli indirizzi scendono

```c
void seconda(int b) {
    int var2 = 200;
    printf("seconda  var2 %p   b %p\n", (void *)&var2, (void *)&b);
}
```

```
$ make bin/stack && ./bin/stack
main     var_main 0x16d366248
main     array[0] 0x16d366250  array[4] 0x16d366260
prima    var1 0x16d366218   a 0x16d36621c
seconda  var2 0x16d3661e8   b 0x16d3661ec
terza    var3 0x16d3661b8   c 0x16d3661bc
prima    var1 vale ancora 100
```

Ogni chiamata **scende** di una cinquantina di byte: il suo *frame*. Gli array crescono verso l'alto, lo stack verso il basso.

# La locale restituita per indirizzo

```c
int *crea(void) {
    int x = 42;
    return &x;          // warning: address of stack memory associated with local variab
}

int *p = crea();        // punta a un frame che non c'è più
printf("%d\n", *p);     // 42, oppure spazzatura, oppure crash: dipende da chi ha chiama
```

- `x` muore con `crea`; `p` è un puntatore **pendente** (*dangling*): punta a memoria che qualcun altro riuserà
- il compilatore lo vede in questo caso semplice; non lo vede quando l'indirizzo passa per una struct o per un parametro
- la soluzione è lo heap: chiedere memoria che **sopravvive alla funzione**

# `malloc` e `free`

```c
#include <stdlib.h>
int *v = malloc(n * sizeof *v);     // n int, non inizializzati
if (v == NULL) { fprintf(stderr, "memoria esaurita\n"); return 1; }   // può finire
for (size_t i = 0; i < n; i++) v[i] = 0;
free(v);                            // una volta sola, lo stesso indirizzo
v = NULL;                           // così un secondo free o un uso sono innocui
```

- `malloc(byte)` restituisce un `void *`, assegnabile a qualunque puntatore senza cast
- `sizeof *v` prende la dimensione dal tipo di `v`; `free` non azzera il puntatore: `v = NULL` lo fate voi

# `calloc` e `realloc`

```c
int *v = calloc(n, sizeof *v);              // n elementi, tutti a zero

int *tmp = realloc(v, 2 * n * sizeof *v);   // il doppio, con i vecchi dati copiati
if (tmp == NULL) { free(v); return 1; }     // v è ancora valido: liberarlo
v = tmp;
```

- `calloc` azzera; `malloc` no, e leggere memoria non inizializzata è *undefined behavior*
- `realloc` può **spostare** il blocco: il vecchio indirizzo non vale più, si usa il risultato
- `v = realloc(v, …)` è la trappola: se fallisce restituisce `NULL` e il blocco si perde

> Il raddoppio con `realloc` è esattamente ciò che fa `ArrayList` quando `add` non ha più posto. Lo scriveremo per intero fra una settimana.

# Chi libera

**JAVA**

```java
int[] v = new int[n];
// ... uso ...
// nessuno: quando non ci sono più riferimenti
// il garbage collector la recupera
```

Il GC segue i riferimenti dallo stack e dalle statiche: ciò che non raggiunge, lo libera. Costo: un thread in più e pause che non decidete voi

**C**

```c
int *v = malloc(n * sizeof *v);
// ... uso ...
free(v);
v = NULL;
```

Nessuno conta i riferimenti: **chi alloca decide chi libera**, e lo scrive nel contratto della funzione. «L'array è del chiamante» è una frase da documentare

# I quattro errori

| Errore | Che cosa succede | Sintomo |
|---|---|---|
| **leak** | `malloc` senza `free` | la memoria cresce, il programma rallenta e muore dopo ore |
| **dangling** | si usa un blocco dopo `free` (*use-after-free*) | valori che cambiano da soli, crash altrove |
| **double free** | `free` due volte sullo stesso indirizzo | corruzione dello heap, `abort` |
| **overflow** | si scrive oltre la fine del blocco | la variabile accanto cambia, crash alla `free` successiva |

> Nessuno dei quattro è un errore di compilazione, e spesso nemmeno di esecuzione: il programma **sembra funzionare**. Il sintomo compare lontano dalla causa, nel codice sbagliato.

# AddressSanitizer li trova

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=address -o errori errori.c
```

```
$ ./bin/errori overflow
==68516==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x602000000100
WRITE of size 4 at 0x602000000100 thread T0
    #0 0x000100fe8d5c in overflow errori.c:31
    #1 0x000100fe8a20 in main errori.c:44
0x602000000100 is located 0 bytes after 16-byte region
$ ./bin/errori dangling
==68519==ERROR: AddressSanitizer: heap-use-after-free on address 0x6020000000f0
READ of size 4 at 0x6020000000f0 thread T0
    #0 0x000102ae8c4c in dangling errori.c:18
```

Il compilatore strumenta ogni accesso alla memoria: il programma gira più lento, ma l'errore esce **con file e riga**. `make asan` in ogni cartella del corso. Funziona con gcc e clang su Linux e macOS, e su Windows dentro WSL (Visual Studio ha `/fsanitize=address`, MinGW no). Il leak lo trova solo su Linux: su macOS c'è `leaks`,

# Esercizio 3 · Un array grande quanto serve

Aprite `LezC01-Dinamico/main.c`, poi `make run` e inserite i numeri.

1. `int *leggi_interi(size_t n)`: alloca `n` interi, controlla `NULL`, li legge con `scanf`, restituisce l'array
2. `somma` e `massimo` su `(const int *v, size_t n)`
3. il `TODO` in fondo al `main`: chi libera `v`? E che cosa deve valere `v` dopo?

> Dieci minuti. Poi `make asan && make run`: se avete dimenticato qualcosa, ve lo dice lui.

# 07

# struct, union e campi di bit



# Una `struct`, e il `typedef` che la rende un tipo

```c
struct rectangle { double x, y, w, h; };          // il tipo si chiama "struct rectangle
struct rectangle r1 = {0, 0, 3, 2};

typedef struct { double x, y, w, h; } Rectangle;  // ora si chiama Rectangle
Rectangle r2 = { .w = 3, .h = 2 };                // inizializzatori designati: x e y a
r2.w = 4;
```

- una `struct` è un blocco di campi **contigui**: la parte dati di una classe, senza metodi
- `typedef` dà un nome al tipo; con gli **inizializzatori designati** l'ordine non conta e i campi omessi valgono zero
- una struct **si copia per intero** con `=`: `r1 = r2` copia quattro `double`

# `.` e `->`, per valore e per puntatore

```
double area_v(Rectangle r)         { return r.w * r.h; }     // copia 32 byte
double area_p(const Rectangle *r)  { return r->w * r->h; }   // copia 8 byte: l'indiriz
void   trasla(Rectangle *r, double dx, double dy) { r->x += dx; r->y += dy; }

Rectangle r = { .w = 3, .h = 2 };
trasla(&r, 1, 1);                  // modifica r
area_p(&r);                        // legge r: const lo promette
```

- `r->w` è `(*r).w`: il campo della struct **puntata**
- per puntatore quando va **modificata** o è grande, `const *` quando va solo letta, per valore se è piccola
- in Java passate sempre il riferimento; in C la scelta è vostra, e si legge nella firma

# Il bimondo in C

**JAVA**

```java
public class Rectangle extends Shape2d {
    private double w, h;
    public double area() { return w * h; }
    public void translate(double dx,
                          double dy) {
        x += dx; y += dy;
    }
}
r.translate(1, 1);
```

Lo stato e i metodi stanno insieme; `this` è implicito. L'ereditarietà da `Shape2d` dà `area` a tutte le figure

**C**

```c
typedef struct {
    double x, y, w, h;
} Rectangle;
double rectangle_area(const Rectangle *r) {
    return r->w * r->h;
}
void rectangle_translate(Rectangle *r,
                         double dx, double dy)
    r->x += dx; r->y += dy;
}
rectangle_translate(&r, 1, 1);
```

Funzioni con il **prefisso del tipo** e la struct come primo parametro: `this` esplicito. L'`area` comune a tutte le figure arriva con i puntatori a funzione

# Struct restituita, array di struct

```c
Rectangle rectangle_new(double w, double h) {
    return (Rectangle){ .w = w, .h = h };        // un letterale composto, restituito pe
}

Rectangle rs[3] = { rectangle_new(1, 1), rectangle_new(2, 3), rectangle_new(4, 1) };
Rectangle *ds = malloc(n * sizeof *ds);          // n rettangoli sullo heap
rs[1].w = 5;                                     // array statico
ds[1].w = 5;                                     // array dinamico: identico, (ds + 1)->
```

- restituire una struct per valore è normale: niente `malloc` per un `Punto` o un `Rectangle`
- un array di struct è un blocco contiguo; `ds[i].campo` vale per l'array e per il puntatore

# La struct che contiene un puntatore

```c
typedef struct {
    char *nome;        // chi possiede la stringa?
    int *voti;         // e questo array?
} Studente;

Studente a = studente_new("Greta", 3);
Studente b = a;        // copia i puntatori, non ciò che puntano: due struct, un solo ar
```

- la copia è **superficiale**: `b.voti` e `a.voti` sono lo stesso blocco. Se si libera `a`, `b` pende
- serve un contratto: `studente_new` alloca, `studente_free` libera, nessun altro
- in C ogni puntatore in una struct chiede: **di chi è?** In Java nessuno libera, e la domanda non c'è

# Esercizio 4 · `Punto` e `Segmento`

Aprite `LezC01-Segmento/main.c`, poi `make run`.

```c
typedef struct { double x, y; } Punto;
typedef struct { Punto a, b; } Segmento;
```

1. `double lunghezza(const Segmento *s)`: `sqrt` sta in `math.h`
2. `Punto punto_medio(const Segmento *s)`: restituito **per valore**
3. `void trasla(Segmento *s, double dx, double dy)`: sposta entrambi gli estremi

> Dieci minuti. `s->a.x` è la `x` del primo estremo: una freccia per il puntatore, un punto per la struct contenuta.

# `union`: gli stessi byte, più letture

```c
#include <stdint.h>
union int_decoder {
    int32_t  valore;         // tutti e tre i campi partono dallo stesso indirizzo
    uint32_t senza_segno;
    uint8_t  byte[4];
};                           // sizeof: 4, il campo più grande
```

- in una `struct` i campi stanno **uno dopo l'altro**, in una `union` **uno sopra l'altro**: la stessa memoria con occhi diversi
- `stdint.h`: `int` è «almeno 16 bit»; quando la larghezza conta si scrive `int32_t`, `uint8_t`. Dimensioni e indici: `size_t`

# `lez.c` dal vivo: -1234 nei suoi byte

```
$ ./bin/lez
valore       = -1234
senza segno  = 4294966062  (hex FFFFFB2E)
byte[0]      = 2E
byte[1]      = FB
byte[2]      = FF
byte[3]      = FF
sizeof(union int_decoder) = 4
```

- **complemento a due**: -1234 è `FFFFFB2E`, cioè $2^{32} - 1234$. Letto senza segno, è un numero enorme: il bit alto non cambia, cambia l'interpretazione
- **endianness**: `byte[0]` è `2E`, il byte **meno** significativo: la macchina è *little-endian* (x86 e ARM). In rete i byte viaggiano *big-endian*, e Java scrive sempre così: ecco perché `DataOutputStream` e il C si capiscono solo dopo una conversione

# Campi di bit, e `sizeof` che non torna

```
struct header {
    uint32_t errore   : 1;      // un bit
    uint32_t tipo     : 3;      // tre bit: da 0 a 7
    uint32_t priorita : 1;
};                              // sizeof: 4, cinque bit in un uint32_t
struct disordinata { char c; int32_t i; char d; };   // sizeof: 12, non 6
struct ordinata    { int32_t i; char c; char d; };   // sizeof: 8
```

- i **campi di bit** si usano come campi normali, `h.tipo = 5`; l'ordine dei bit lo decide il compilatore: per file e rete, maschere e shift
- il **padding**: ogni campo sta a un indirizzo multiplo della sua dimensione. Campi dal più grande al più piccolo

# Esercizio 5 · I byte di un intero, tre flag in un byte

Aprite `LezC01-Byte/main.c`, poi `make run`.

1. `void scomponi(uint32_t v, uint8_t byte[4])`: dal meno al più significativo, con `>>` e `& 0xFF`
2. `void scomponi_union(uint32_t v, uint8_t byte[4])`: la stessa cosa leggendo la memoria con una `union`. I due risultati coincidono? Su quali macchine?
3. `flag_set`, `flag_clear`, `flag_test` su un `uint8_t` con `FLAG_ERRORE`, `FLAG_URGENTE`, `FLAG_LETTO`: `|`, `& ~`, `&`

> Dieci minuti. `0xCAFEBABE` deve diventare `BE BA FE CA`: è la firma di ogni file `.class`, e la ritroveremo leggendone uno byte per byte.

# 08

# Preprocessore e macro



# Prima del compilatore

```
hello.c  →  preprocessore  →  compilatore  →  assemblatore  →  linker  →  hello
            #include, #define      .s                .o          + libc
```

- il primo dei quattro passi **lavora sul testo**: incolla i file inclusi, sostituisce le macro, toglie i rami `#if` falsi. Non conosce il C
- `#include <stdio.h>` cerca nelle cartelle di sistema; `#include "rettangolo.h"` prima nella cartella del sorgente
- `gcc -E hello.c` si ferma qui e mostra che cosa vede davvero il compilatore: decine di migliaia di righe per un `printf`. Uguale su macOS, Linux e Windows

> `make pre` in `LezC01-Esempi` stampa il `main` di `macro.c` dopo l'espansione: le prossime quattro slide, viste dal preprocessore.

# `#define`, e le parentesi che mancano

```c
#define N 100                       // macro oggetto: N diventa 100 ovunque
#define SQUARE(x) x * x             // macro funzione: sostituzione testuale

SQUARE(a + 1)                       // → a + 1 * a + 1   : con a = 2 vale 5, non 9
```

```c
#define SQUARE(x) ((x) * (x))       // → ((a + 1) * (a + 1))
```

- la macro non conosce la precedenza: il testo del parametro entra nel testo della macro così com'è
- **ogni parametro fra parentesi, e tutta la macro fra parentesi**: sempre
- per le costanti meglio `enum { N = 100 };`: ha un tipo e si vede nel debugger

# La doppia valutazione

```c
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int i = 1, j = 5;
int m = MAX(i++, j);     // → ((i++) > (j) ? (i++) : (j)) : m = 5, i = 2
int m = MAX(i, j++);     // j incrementato due volte: m = 6, j = 7
```

- il parametro compare **due volte** nel testo espanso: due valutazioni, due effetti collaterali. `MAX(leggi(), 0)` legge due volte
- una funzione non ha il problema, e con `static inline` il compilatore la espande comunque

> Regola: mai un'espressione con effetti collaterali come argomento di una macro, e mai una macro funzione dove basta una funzione.

# Più istruzioni: le graffe non bastano

```
#define SWAP(a, b) tmp = a; a = b; b = tmp

if (x > y)
    SWAP(x, y);          // → if (x > y) tmp = x; x = y; y = tmp;
```

Solo `tmp = x` sta sotto l'`if`. Allora un blocco:

```
#define SWAP(a, b) { int tmp_ = (a); (a) = (b); (b) = tmp_; }
if (x > y) SWAP(x, y);   // → if (x > y) { ... };   il ; chiude l'if
else ...                 // error: expected expression before 'else'
```

Il `;` dopo `SWAP(x, y)` chiude l'`if` e l'`else` resta orfano. Senza `else` funziona: il bug aspetta.

# `do { } while (0)`, e il tipo come parametro

```c
#define SWAP(a, b) do { int tmp_ = (a); (a) = (b); (b) = tmp_; } while (0)
if (x > y) SWAP(x, y); else ...     // un'istruzione sola: il ; è quello del while
```

```c
#define SWAP_T(T, a, b) do { T tmp_ = (a); (a) = (b); (b) = tmp_; } while (0)

SWAP_T(double, r.w, r.h);
SWAP_T(Rectangle *, p, q);
```

- `do { } while (0)` è **una** istruzione che pretende il `;`: sta sotto un `if`, prima di un `else`, ovunque
- `SWAP` scambia solo `int`: il **tipo come parametro** la rende generica. Il C non ha `<T>`: questo è il suo modo

# Compilazione condizionale e il `LOG` che si spegne

```c
#ifndef RETTANGOLO_H                    // include guard: il file si può includere due volt
#define RETTANGOLO_H
#endif

#ifdef DEBUG
#define LOG(msg) fprintf(stderr, "[%s:%d] %s\n", __FILE__, __LINE__, msg)
#else
#define LOG(msg) ((void)0)              // sparisce: zero costo nel programma finale
#endif
```

- `#ifdef DEBUG` si accende con `gcc -DDEBUG`: lo stesso sorgente, due programmi
- `__FILE__` e `__LINE__` diventano file e riga **della chiamata**: un log che dice dove

# Esercizio 6 · Macro che mentono

Aprite `LezC01-Macro/main.c`, poi `make run`: compila e stampa valori sbagliati. Si aggiustano le macro, non il `main`.

1. `SQUARE(3 + 1)` deve valere 16
2. `LOG` deve sparire con `make` e stampare file e riga con `make debug` (che passa `-DDEBUG`)
3. se avanza tempo, `SWAP` dentro un `if` senza graffe: `do { } while (0)`, poi la variante `SWAP_T(T, a, b)`

> Cinque minuti. Per vedere che cosa avete scritto davvero: `gcc -E -P main.c | grep -A3 'int main'`.

# 09

# In pratica



# Errori frequenti

- **puntatore non inizializzato**: `int *p; *p = 3;` compila e scrive a caso. Sempre `= NULL` o `= &x`
- `scanf("%d", n)` senza `&`: riceve il valore di `n` e scrive a quell'indirizzo
- `sizeof(a)` **dentro la funzione**: misura il puntatore. La lunghezza si passa
- `a == b` fra stringhe: confronta gli indirizzi. `strcmp(a, b) == 0`
- `return &locale`: il frame muore con la funzione. `malloc`, o una struct per valore
- `v = realloc(v, …)`: se fallisce il blocco è perso. Variabile d'appoggio
- `free` **dimenticato o doppio**: `make asan` prima di dire «funziona»
- **macro senza parentesi**, o con `i++` come argomento

# Il C in due righe

Ogni byte ha un indirizzo, e ogni blocco ha **un proprietario** che lo libera.

Il compilatore fa quello che avete scritto, **non quello che intendevate**: `-Wall -Wextra -Werror` e ASan sono i vostri alleati, non ostacoli.

> La prossima volta: la lambda diventa un **puntatore a funzione**, la tabella dei comandi della chat torna in C tale e quale, `qsort` la scriviamo noi, e vediamo che cosa c'è davvero sotto un oggetto Java.